```
import { useState, useEffect, useCallback, useRef } from "react";

// ── DESIGN TOKENS ──────────────────────────────────────────────────────
const C = {
  navy:      "#0d2137",
  navyMid:   "#1a3a5c",
  navyLight: "#234d7a",
  teal:      "#2e7d8c",
  tealLight: "#3a9aad",
  slate:     "#4a6580",
  bg:        "#f2f5f8",
  bgCard:    "#ffffff",
  border:    "#dde3ea",
  text:      "#0d1f2d",
  textMid:   "#4a6580",
  textLight: "#7a92a8",
  white:     "#ffffff",
  success:   "#2e7d5a",
  danger:    "#8b2635",
  warning:   "#8b6914",
};

const LEVEL_CONFIG = {
  A: { color: "#8b2635", bg: "#f9f2f3", label: "Level A", desc: "Non-stand / Bed
  B: { color: "#8b6914", bg: "#f9f6ee", label: "Level B", desc: "Seated + Brief S
  C: { color: "#234d7a", bg: "#f0f4f9", label: "Level C", desc: "Seated + Standin
  D: { color: "#2e7d5a", bg: "#f0f6f3", label: "Level D", desc: "Higher Function
};

const EXERCISE_BANK = {
  A: {
    upper: ["Postural alignment & scapular retraction","Shoulder flexion (assiste
    lower: ["Ankle pumps & circles","Seated marching","Seated knee extension","Hi
  },
  B: {
    upper: ["Seated band row","Seated band chest press","Shoulder flexion w/ ligh
    lower: ["Seated marching (progressed)","Seated knee extension w/ light load",
  },
  C: {
    upper: ["Band row (seated or standing)","Chest press (seated or supported sta
    lower: ["Sit-to-stand","Standing marching","Standing heel raises","Mini-squat
  },
  D: {
    upper: ["Standing row & chest press","Overhead / near-overhead press","Functi
```

```
    lower: ["Sit-to-stand (performance focus)","Step-up or progressed step-tap","
  },
};

const METRICS = [
  { key: "restingHR",      label: "Resting HR (bpm)",         type: "number" },
  { key: "exerciseHR",     label: "Exercise HR (bpm)",        type: "number" },
  { key: "bpPre",          label: "BP Pre-Exercise",          type: "text",   pla
  { key: "bpPost",         label: "BP Post-Exercise",         type: "text",   pla
  { key: "spo2",           label: "SpO₂ (%)",                 type: "number" },
  { key: "rpe",            label: "RPE (6–20)",               type: "number" },
  { key: "painPre",        label: "Pain Pre (0–10)",          type: "number" },
  { key: "painPost",       label: "Pain Post (0–10)",         type: "number" },
  { key: "sitToStand",     label: "Sit-to-Stand (30s reps)",  type: "number" },
  { key: "gaitDistance",   label: "Gait Distance",            type: "text",   pla
  { key: "gripStrength",   label: "Grip Strength (kg)",       type: "number" },
  { key: "bergBalance",    label: "Berg Balance (0–56)",      type: "number" },
  { key: "sittingTol",     label: "Sitting Tolerance (min)",  type: "number" },
  { key: "sessionDur",     label: "Session Duration (min)",   type: "number" },
  { key: "mood",           label: "Patient Mood (1–5)",       type: "number" },
  { key: "participation",  label: "Participation",            type: "select", opt
  { key: "fallHistory",    label: "Falls Since Admission",    type: "number" },
];

const ROLES = { ADMIN: "admin", SUPERVISOR: "supervisor", EP: "ep" };

const DEFAULT_USERS = [
  { id:"u1", username:"admin",      password:"admin123",  role:ROLES.ADMIN,
  { id:"u2", username:"supervisor", password:"super123",  role:ROLES.SUPERVISOR,
  { id:"u3", username:"ep",         password:"ep123",     role:ROLES.EP,
];
const DEFAULT_SITES = [{ id:"s1", name:"Hackensack", address:"Hackensack, NJ" }];
const INACTIVITY_MS = 10 * 60 * 1000;
const WARN_MS       =  9 * 60 * 1000;

// ── STORAGE ──────────────────────────────────────────────────────────
async function load(key) { try { const r = await window.storage.get(key); return
async function save(key, val) { try { await window.storage.set(key, JSON.stringif

async function audit(user, action, detail="") {
  const entry = { id: Date.now().toString(), ts: new Date().toISOString(), userId
  const existing = await load("kin_audit") || [];
  await save("kin_audit", [entry, ...existing].slice(0, 500));
}

// ── PRINT REPORT ─────────────────────────────────────────────────────
function printReport(patient, sessions, user, sites) {
```

```
  const sorted = [...sessions].sort((a,b) => a.date.localeCompare(b.date));
  const lc = LEVEL_CONFIG[patient.level];
  const site = sites.find(s => s.id === patient.siteId);
  const levelColors = { A:"#8b2635", B:"#8b6914", C:"#234d7a", D:"#2e7d5a" };
  const levelHistory = sorted.reduce((acc,s) => {
    if (!acc.length || acc[acc.length-1].level !== s.level) acc.push({ date:s.dat
    return acc;
  }, []);

  const html = `<!DOCTYPE html><html><head><meta charset="UTF-8"><title>Kinesis —
<style>
*{box-sizing:border-box;margin:0;padding:0}
body{font-family:'Segoe UI',Arial,sans-serif;font-size:11px;color:#0d1f2d;backgro
.wm{position:fixed;top:50%;left:50%;transform:translate(-50%,-50%) rotate(-35deg)
.ct{position:relative;z-index:1}
.conf{background:#0d2137;color:#fff;text-align:center;padding:5px;font-size:9px;f
.hdr{display:flex;justify-content:space-between;align-items:flex-start;padding-bo
.brand{font-size:20px;font-weight:800;color:#0d2137;letter-spacing:-0.5px}
.brand span{color:#2e7d8c}
.stitle{font-size:8px;font-weight:700;text-transform:uppercase;letter-spacing:1.5
.igrid{display:grid;grid-template-columns:1fr 1fr 1fr;gap:7px;margin-bottom:10px}
.iitem label{display:block;font-size:7px;font-weight:700;text-transform:uppercase
.iitem span{font-size:11px;color:#0d2137;font-weight:600}
.lbadge{display:inline-flex;align-items:center;gap:4px;padding:2px 8px;border-rad
.dbox{background:#f2f5f8;border-left:3px solid #0d2137;padding:7px 10px;border-ra
table{width:100%;border-collapse:collapse;font-size:10px;margin-bottom:10px}
th{background:#0d2137;color:#fff;padding:5px 7px;text-align:left;font-size:8px;te
td{padding:5px 7px;border-bottom:1px solid #f0f3f6;vertical-align:top}
tr:nth-child(even) td{background:#f8f9fb}
.sblock{border:1px solid #dde3ea;border-radius:5px;margin-bottom:8px;page-break-i
.shdr{background:#f2f5f8;padding:6px 10px;display:flex;justify-content:space-betw
.sbody{padding:8px 10px}
.mgrid{display:grid;grid-template-columns:repeat(4,1fr);gap:4px;margin-bottom:7px
.mchip{background:#f2f5f8;border-radius:3px;padding:4px 6px}
.mchip .mk{font-size:7px;color:#7a92a8;text-transform:uppercase}
.mchip .mv{font-size:11px;font-weight:700;color:#0d2137}
.nbox{background:#fdfbf5;border-left:2px solid #8b6914;padding:5px 8px;font-size:
.gbox{background:#f5fbf7;border-left:2px solid #2e7d5a;padding:5px 8px;font-size:
.ft{margin-top:18px;padding-top:8px;border-top:1px solid #dde3ea;display:flex;jus
.sig{margin-top:28px;display:grid;grid-template-columns:1fr 1fr;gap:40px}
.sf{border-top:1px solid #333;padding-top:3px;font-size:8px;color:#555}
@media print{body{padding:14px}}
</style></head><body>
<div class="wm">CONFIDENTIAL</div>
<div class="ct">
<div class="conf">🔒 CONFIDENTIAL — PROTECTED HEALTH INFORMATION — AUTHORIZED CLIN
<div class="hdr">
```

```
  <div>
    <div class="brand">Kinesis <span>Clinical</span></div>
    <div style="font-size:9px;color:#4a6580;margin-top:2px;">Exercise Physiology
    ${site?`<div style="font-size:9px;color:#7a92a8;margin-top:2px;">${site.name}
    <div style="margin-top:8px;font-size:16px;font-weight:700;color:#0d2137">${pa
    <div style="font-size:10px;color:#4a6580;">Patient Progress Report</div>
  </div>
  <div style="text-align:right;font-size:9px;color:#7a92a8;">
    <div>Generated: ${new Date().toLocaleDateString("en-US",{year:"numeric",month
    <div>By: ${user?.name||user?.username} (${user?.role})</div>
    <div style="margin-top:6px;">Current Level: <span class="lbadge"><span class=
    <div style="margin-top:2px;">Total Sessions: <strong>${sorted.length}</strong
  </div>
</div>

<div class="stitle">Patient Information</div>
<div class="igrid">
  <div class="iitem"><label>Date of Birth</label><span>${patient.dob||"—"}</span>
  <div class="iitem"><label>Room / Unit</label><span>${patient.room||"—"}</span><
  <div class="iitem"><label>Admission Date</label><span>${patient.admitDate||"—"}
  <div class="iitem"><label>Attending Physician</label><span>${patient.physician|
  <div class="iitem"><label>Functional Level</label><span class="lbadge" style="b
  <div class="iitem"><label>Site</label><span>${site?.name||"—"}</span></div>
</div>
${patient.diagnosis?`<div class="dbox"><strong>Diagnosis / History:</strong> ${pa
${patient.notes?`<div class="dbox" style="border-left-color:#2e7d8c;"><strong>Not

${levelHistory.length>1?`
<div class="stitle">Functional Level Progression</div>
<div style="display:flex;gap:8px;flex-wrap:wrap;margin-bottom:10px;align-items:ce
${levelHistory.map((lh,i)=>`${i>0?'<span style="color:#aaa;font-size:12px;">→</sp
</div>`:""}

${sorted.length>0?`
<div class="stitle">Session Summary</div>
<table><thead><tr><th>Date</th><th>Level</th><th>Clinician</th><th>Exercises</th>
<tbody>${sorted.map(s=>`<tr><td>${s.date}</td><td><span class="lbadge" style="fon

<div class="stitle">Detailed Session Log</div>
${sorted.map((s,idx)=>{
  const fm=METRICS.filter(m=>s.metrics[m.key]&&s.metrics[m.key]!=="N/A"&&s.metric
  return `<div class="sblock">
<div class="shdr"><div style="font-weight:700;color:#0d2137;font-size:11px;">Sess
<div style="font-size:10px;color:#4a6580;">Clinician: <strong>${s.clinician}</str
<div class="sbody">
${fm.length>0?`<div class="mgrid">${fm.map(m=>`<div class="mchip"><div class="mk"
${s.exercises.length>0?`<table><thead><tr><th>Exercise</th><th>Sets</th><th>Reps
```

```
${s.sessionNotes?`<div class="nbox"><strong>Notes:</strong> ${s.sessionNotes}</di
${s.nextGoals?`<div class="gbox"><strong>Next Goals:</strong> ${s.nextGoals}</div
</div></div>`;}).join("")}`:"<p style='color:#7a92a8;padding:10px 0;'>No sessions

<div class="sig"><div class="sf">Clinician Signature / Date</div><div class="sf">
<div class="ft">
  <span>Kinesis Clinical — Exercise Physiology Management Platform${site?" — "+si
  <span>CONFIDENTIAL — PHI — Authorized Use Only</span>
  <span>Generated ${new Date().toLocaleString()} by ${user?.username}</span>
</div>
</div></body></html>`;

  const win = window.open("","_blank");
  win.document.write(html);
  win.document.close();
  setTimeout(()=>win.print(),500);
}

// ── UI PRIMITIVES ──────────────────────────────────────────────────
const inputStyle = { width:"100%", padding:"9px 11px", border:`1.5px solid ${C.bo
const labelStyle = { display:"block", fontSize:11, fontWeight:600, color:C.textMi

function Field({ label, children, required }) {
  return <div style={{ marginBottom:14 }}><label style={labelStyle}>{label}{requi
}
function Input({ label, value, onChange, type="text", placeholder="", required=fa
  return <Field label={label} required={required}><input type={type} value={value
}
function Select({ label, value, onChange, options, required=false }) {
  return <Field label={label} required={required}>
    <select value={value} onChange={e=>onChange(e.target.value)} style={{...input
      {options.map(o=>typeof o==="string"?<option key={o} value={o}>{o}</option>:
    </select>
  </Field>;
}
function Textarea({ label, value, onChange, placeholder="", rows=3 }) {
  return <Field label={label}><textarea value={value} onChange={e=>onChange(e.tar
}

function Btn({ children, onClick, variant="primary", size="md", style={}, disable
  const vs = {
    primary:   { background:C.navyMid,  color:"#fff", border:"none" },
    secondary: { background:"#fff",     color:C.navyMid, border:`1.5px solid ${C.
    teal:      { background:C.teal,     color:"#fff", border:"none" },
    danger:    { background:C.danger,   color:"#fff", border:"none" },
    success:   { background:C.success,  color:"#fff", border:"none" },
    ghost:     { background:"transparent", color:C.navyMid, border:`1.5px solid $
```

```
    print:     { background:"#3a4a5a",  color:"#fff", border:"none" },
  };
  const ss = { sm:{padding:"5px 12px",fontSize:12}, md:{padding:"9px 16px",fontSi
  return <button onClick={onClick} disabled={disabled}
    style={{ borderRadius:5, cursor:disabled?"not-allowed":"pointer", fontWeight:
}

function LevelTag({ level, size="sm" }) {
  const lc = LEVEL_CONFIG[level];
  const fs = size==="lg" ? 13 : 11;
  const dot = size==="lg" ? 9 : 7;
  return (
    <span style={{ display:"inline-flex", alignItems:"center", gap:5, background:
      <span style={{ width:dot, height:dot, borderRadius:"50%", background:lc.col
      {lc.label}
    </span>
  );
}

function Card({ children, style={} }) {
  return <div style={{ background:C.bgCard, borderRadius:6, border:`1px solid ${C
}

function SectionTitle({ children }) {
  return <div style={{ fontSize:10, fontWeight:700, textTransform:"uppercase", le
}

function DataTable({ headers, rows, emptyMsg="No data." }) {
  return (
    <div style={{ overflowX:"auto" }}>
      <table style={{ width:"100%", borderCollapse:"collapse", fontSize:13 }}>
        <thead>
          <tr style={{ background:C.navy }}>
            {headers.map(h=><th key={h} style={{ padding:"9px 12px", textAlign:"l
          </tr>
        </thead>
        <tbody>
          {rows.length===0
            ?<tr><td colSpan={headers.length} style={{ padding:"20px 12px", color
            :rows.map((row,i)=><tr key={i} style={{ borderBottom:`1px solid ${C.b
              {row.map((cell,j)=><td key={j} style={{ padding:"9px 12px", color:C
            </tr>)
          }
        </tbody>
      </table>
    </div>
  );
```

```
}

function StatCard({ label, value, sub }) {
  return (
    <div style={{ background:C.bgCard, border:`1px solid ${C.border}`, borderRadi
      <div style={{ fontSize:28, fontWeight:800, color:C.navy, lineHeight:1, marg
      <div style={{ fontSize:11, fontWeight:600, color:C.textMid, textTransform:"
      {sub&&<div style={{ fontSize:11, color:C.textLight, marginTop:2 }}>{sub}</d
    </div>
  );
}

// ── LOGIN ──────────────────────────────────────────────────────────────
function Login({ onLogin, error }) {
  const [u,setU]=useState(""); const [p,setP]=useState("");
  return (
    <div style={{ minHeight:"100vh", background:C.navy, display:"flex", alignItem
      <div style={{ background:"#fff", borderRadius:8, padding:"40px 32px", width
        <div style={{ textAlign:"center", marginBottom:32 }}>
          <div style={{ fontSize:28, fontWeight:800, color:C.navy, letterSpacing:
          <div style={{ fontSize:12, color:C.textLight, marginTop:4, letterSpacin
          <div style={{ width:40, height:2, background:C.teal, margin:"12px auto
        </div>
        {error&&<div style={{ background:"#fdf2f3", border:`1px solid ${C.danger}
        <Input label="Username" value={u} onChange={setU} placeholder="Enter user
        <Input label="Password" value={p} onChange={setP} type="password" placeho
        <Btn onClick={()=>onLogin(u,p)} variant="primary" size="lg" style={{width
        <div style={{ marginTop:20, padding:"10px 12px", background:C.bg, borderR
          🔒 This system contains Protected Health Information.<br/>Unauthorized
        </div>
      </div>
    </div>
  );
}

// ── INACTIVITY ─────────────────────────────────────────────────────────
function InactivityGuard({ onLogout, children }) {
  const [warn,setWarn]=useState(false);
  const wt=useRef(null); const lt=useRef(null);
  const reset=useCallback(()=>{
    setWarn(false); clearTimeout(wt.current); clearTimeout(lt.current);
    wt.current=setTimeout(()=>setWarn(true),WARN_MS);
    lt.current=setTimeout(()=>onLogout("timeout"),INACTIVITY_MS);
  },[onLogout]);
  useEffect(()=>{
    const ev=["mousedown","mousemove","keydown","touchstart","scroll"];
    ev.forEach(e=>window.addEventListener(e,reset)); reset();
```

```
    return()=>{ ev.forEach(e=>window.removeEventListener(e,reset)); clearTimeout(
  },[reset]);
  return (<>
    {warn&&<div style={{ position:"fixed",top:0,left:0,right:0,zIndex:9999,backgr
      <span style={{fontWeight:600,fontSize:14}}>⚠️ Session will auto-logout in 1
      <Btn onClick={reset} size="sm" style={{background:"#fff",color:C.warning,bo
    </div>}
    {children}
  </>);
}

// ── PATIENT FORM ──────────────────────────────────────────────────────
function PatientForm({ initial, onSave, onCancel, currentUser }) {
  const INIT={id:"",name:"",dob:"",diagnosis:"",admitDate:"",room:"",physician:""
  const [f,setF]=useState(initial||INIT);
  const s=k=>v=>setF(x=>({...x,[k]:v}));
  return (
    <Card>
      <SectionTitle>{initial?.id?"Edit Patient":"New Patient"}</SectionTitle>
      <div style={{display:"grid",gridTemplateColumns:"repeat(auto-fill,minmax(20
        <Input label="Full Name" value={f.name} onChange={s("name")} required/>
        <Input label="Date of Birth" value={f.dob} onChange={s("dob")} type="date
        <Input label="Room / Unit" value={f.room} onChange={s("room")}/>
        <Input label="Admission Date" value={f.admitDate} onChange={s("admitDate"
        <Input label="Attending Physician" value={f.physician} onChange={s("physi
        <Select label="Functional Level" value={f.level} onChange={s("level")} op
      </div>
      <Textarea label="Primary Diagnosis / Medical History" value={f.diagnosis} o
      <Textarea label="Additional Notes" value={f.notes} onChange={s("notes")} ro
      <div style={{display:"flex",gap:8,marginTop:4,flexWrap:"wrap"}}>
        <Btn onClick={()=>{if(!f.name)return alert("Name required");onSave({...f,
        <Btn onClick={onCancel} variant="secondary">Cancel</Btn>
      </div>
    </Card>
  );
}

// ── SESSION FORM ──────────────────────────────────────────────────────
function SessionForm({ patient, onSave, onCancel, currentUser }) {
  const INIT={date:new Date().toISOString().split("T")[0],clinician:currentUser.n
  const [f,setF]=useState(INIT);
  const sf=k=>v=>setF(x=>({...x,[k]:v}));
  const sm=k=>v=>setF(x=>({...x,metrics:{...x.metrics,[k]:v}}));
  const tx=ex=>setF(x=>({...x,exercises:x.exercises.includes(ex)?x.exercises.filt
  const sd=(t,ex,v)=>setF(x=>({...x,[t]:{...x[t],[ex]:v}}));
  const bank=EXERCISE_BANK[f.level]||EXERCISE_BANK.A;
```

```
return (
  <div>
    <Card>
      <SectionTitle>Log Session — {patient.name}</SectionTitle>
      <p style={{margin:"0 0 16px",color:C.textLight,fontSize:13}}>Current Leve
      <div style={{display:"grid",gridTemplateColumns:"repeat(auto-fill,minmax(
        <Input label="Session Date" value={f.date} onChange={sf("date")} type="
        <Input label="Clinician" value={f.clinician} onChange={sf("clinician")}
        <Select label="Session Level" value={f.level} onChange={sf("level")} op
      </div>
    </Card>

    <Card>
      <SectionTitle>Vital Signs & Metrics</SectionTitle>
      <div style={{display:"grid",gridTemplateColumns:"repeat(auto-fill,minmax(
        {METRICS.map(m=>m.type==="select"
          ?<Select key={m.key} label={m.label} value={f.metrics[m.key]||m.optio
          :<Input key={m.key} label={m.label} value={f.metrics[m.key]||""} onCh
        )}
      </div>
    </Card>

    <Card>
      <SectionTitle>Exercise Selection — {LEVEL_CONFIG[f.level].label}</Section
      {["upper","lower"].map(cat=>(
        <div key={cat} style={{marginBottom:20}}>
          <div style={{fontSize:11,fontWeight:700,textTransform:"uppercase",let
            {cat==="upper"?"Upper Extremity":"Lower Extremity"}
          </div>
          {(bank[cat]||[]).map(ex=>{
            const sel=f.exercises.includes(ex);
            return (
              <div key={ex} style={{marginBottom:8}}>
                <label style={{display:"flex",alignItems:"center",gap:8,cursor:
                  <input type="checkbox" checked={sel} onChange={()=>tx(ex)} st
                  <span style={{fontSize:13,color:sel?C.text:C.textMid,fontWeig
                </label>
                {sel&&(
                  <div style={{display:"grid",gridTemplateColumns:"repeat(auto-
                    <Input label="Sets" value={f.sets[ex]||""} onChange={v=>sd(
                    <Input label="Reps / Time" value={f.reps[ex]||""} onChange=
                    <Input label="Resistance" value={f.resistance[ex]||""} onCh
                  </div>
                )}
              </div>
            );
          })}
```

```
        </div>
      ))}

      {/* CUSTOM EXERCISES */}
      <div style={{marginTop:4,paddingTop:16,borderTop:`1px solid ${C.border}`}
        <div style={{fontSize:11,fontWeight:700,textTransform:"uppercase",lette
        {(f.customExercises||[]).map((ex,i)=>(
          <div key={i} style={{marginBottom:8}}>
            <div style={{display:"flex",alignItems:"center",gap:8,marginBottom:
              <span style={{fontSize:13,color:C.text,fontWeight:600}}>✦ {ex}</s
              <button onClick={()=>setF(x=>({...x,customExercises:(x.customExer
            </div>
            <div style={{display:"grid",gridTemplateColumns:"repeat(auto-fill,m
              <Input label="Sets" value={f.sets[ex]||""} onChange={v=>sd("sets"
              <Input label="Reps / Time" value={f.reps[ex]||""} onChange={v=>sd
              <Input label="Resistance" value={f.resistance[ex]||""} onChange={
            </div>
          </div>
        ))}
        <div style={{display:"flex",gap:8,alignItems:"flex-end",marginTop:8}}>
          <div style={{flex:1}}>
            <Input label="Add Custom Exercise" value={f._customInput||""} onCha
          </div>
          <div style={{marginBottom:14}}>
            <Btn onClick={()=>{
              const name=(f._customInput||"").trim();
              if(!name)return;
              if(f.exercises.includes(name)||(f.customExercises||[]).includes(n
              setF(x=>({...x,customExercises:[...(x.customExercises||[]),name],
            }} variant="teal" size="sm">+ Add</Btn>
          </div>
        </div>
      </div>
    </Card>

    <Card>
      <SectionTitle>Session Notes</SectionTitle>
      <Textarea label="Notes & Observations" value={f.sessionNotes} onChange={s
      <Textarea label="Goals for Next Session" value={f.nextGoals} onChange={sf
    </Card>

    <div style={{display:"flex",gap:8,marginBottom:24,flexWrap:"wrap"}}>
      <Btn onClick={()=>{if(!f.clinician||!f.date)return alert("Date and clinic
      <Btn onClick={onCancel} variant="secondary" size="lg">Cancel</Btn>
    </div>
  </div>
);
```

```
}

// ── SESSION CARD ──────────────────────────────────────────────────────
function SessionCard({ session, onDelete, canDelete, defaultOpen=false }) {
  const [open,setOpen]=useState(defaultOpen);
  const filled=METRICS.filter(m=>session.metrics[m.key]&&session.metrics[m.key]!=
  return (
    <div style={{border:`1px solid ${C.border}`,borderRadius:6,marginBottom:8,ove
      <div onClick={()=>setOpen(o=>!o)} style={{padding:"10px 14px",background:op
        <div style={{display:"flex",gap:10,alignItems:"center",flexWrap:"wrap"}}>
          <LevelTag level={session.level}/>
          <span style={{fontWeight:600,color:C.text,fontSize:13}}>{session.date}<
          <span style={{color:C.textMid,fontSize:13}}>{session.clinician}</span>
          <span style={{color:C.textLight,fontSize:12}}>{session.exercises.length
        </div>
        <span style={{color:C.textLight,fontSize:11}}>{open?"▲":"▼"}</span>
      </div>
      {open&&(
        <div style={{padding:14,borderTop:`1px solid ${C.border}`}}>
          {filled.length>0&&(
            <div style={{marginBottom:14}}>
              <div style={{fontSize:10,fontWeight:700,textTransform:"uppercase",c
              <div style={{display:"grid",gridTemplateColumns:"repeat(auto-fill,m
                {filled.map(m=>(
                  <div key={m.key} style={{background:C.bg,borderRadius:5,padding
                    <div style={{fontSize:10,color:C.textLight,marginBottom:2}}>{
                    <div style={{fontWeight:700,color:C.text,fontSize:15}}>{sessi
                  </div>
                ))}
              </div>
            </div>
          )}
          {session.exercises.length>0&&(
            <div style={{marginBottom:14,overflowX:"auto"}}>
              <div style={{fontSize:10,fontWeight:700,textTransform:"uppercase",c
              <DataTable
                headers={["Exercise","Sets","Reps / Time","Resistance"]}
                rows={session.exercises.map(ex=>[ex,session.sets[ex]||"—",session
              />
            </div>
          )}
          {session.sessionNotes&&<div style={{marginBottom:10}}><div style={{font
          {session.nextGoals&&<div style={{marginBottom:10}}><div style={{fontSiz
          {canDelete&&<Btn onClick={()=>{if(window.confirm("Delete this session?"
        </div>
      )}
    </div>
```

```
  );
}

// ── PATIENT DETAIL ──────────────────────────────────────────────────
function PatientDetail({ patient, sessions, onBack, onAddSession, onEdit, onDelet
  const [addS,setAddS]=useState(false);
  const [edit,setEdit]=useState(false);
  const sorted=[...sessions].sort((a,b)=>b.date.localeCompare(a.date));
  const canDelete=currentUser.role!==ROLES.EP;
  const lc=LEVEL_CONFIG[patient.level];

  if(addS) return <SessionForm patient={patient} onSave={s=>{onAddSession(s);setA
  if(edit) return <PatientForm initial={patient} onSave={p=>{onEdit(p);setEdit(fa

  return (
    <div>
      <button onClick={onBack} style={{background:"none",border:"none",cursor:"po

      <Card style={{borderTop:`3px solid ${lc.color}`}}>
        <div style={{display:"flex",justifyContent:"space-between",alignItems:"fl
          <div>
            <h2 style={{margin:"0 0 6px",color:C.navy,fontSize:20,fontWeight:700}
            <div style={{display:"flex",gap:14,flexWrap:"wrap",fontSize:13,color:
              {patient.dob&&<span>DOB: {patient.dob}</span>}
              {patient.room&&<span>Room: {patient.room}</span>}
              {patient.physician&&<span>Dr. {patient.physician}</span>}
              {patient.admitDate&&<span>Admitted: {patient.admitDate}</span>}
            </div>
            <div style={{display:"flex",alignItems:"center",gap:10}}>
              <LevelTag level={patient.level} size="lg"/>
              <span style={{fontSize:12,color:C.textLight}}>{lc.desc}</span>
            </div>
          </div>
          <div style={{display:"flex",gap:8,flexWrap:"wrap"}}>
            <Btn onClick={()=>onPrint(patient,sessions)} variant="print" size="sm
            <Btn onClick={()=>setEdit(true)} variant="secondary" size="sm">Edit</
            <Btn onClick={()=>setAddS(true)} variant="primary" size="sm">+ Log Se
          </div>
        </div>
        {patient.diagnosis&&<div style={{marginTop:12,padding:"8px 12px",backgrou
        {patient.notes&&<div style={{marginTop:6,padding:"8px 12px",background:C.
      </Card>

      <Card>
        <SectionTitle>Functional Level</SectionTitle>
        <div style={{display:"flex",gap:8,flexWrap:"wrap"}}>
          {Object.entries(LEVEL_CONFIG).map(([l,c])=>(
```

```
            <button key={l} onClick={()=>onLevelChange(patient.id,l)}
              style={{padding:"8px 16px",borderRadius:5,border:patient.level===l?
              {c.label}
            </button>
          ))}
        </div>
      </Card>

      <div style={{display:"flex",justifyContent:"space-between",alignItems:"cent
        <h4 style={{margin:0,color:C.navy,fontSize:14,fontWeight:600}}>Session Hi
      </div>
      {sorted.length===0
        ?<Card><p style={{margin:0,color:C.textLight,textAlign:"center",fontSize:
        :sorted.map(s=><SessionCard key={s.id} session={s} onDelete={id=>onDelete
      }
    </div>
  );
}

// ── ROSTER ──────────────────────────────────────────────────────────────
function Roster({ patients, sessions, onSelect, onAdd, onPrintAll, currentUser, s
  const [adding,setAdding]=useState(false);
  const [filter,setFilter]=useState("active");
  const [lvl,setLvl]=useState("All");
  const [q,setQ]=useState("");
  const [siteFilter,setSiteFilter]=useState("All");

  const vis=patients.filter(p=>{
    if(filter==="active"&&!p.active)return false;
    if(filter==="inactive"&&p.active)return false;
    if(lvl!=="All"&&p.level!==lvl)return false;
    if(q&&!p.name.toLowerCase().includes(q.toLowerCase()))return false;
    if(currentUser.role===ROLES.ADMIN&&siteFilter!=="All"&&p.siteId!==siteFilter)
    return true;
  });

  if(adding) return <PatientForm onSave={p=>{onAdd(p);setAdding(false);}} onCance

  return (
    <div>
      <div style={{display:"flex",justifyContent:"space-between",alignItems:"cent
        <div>
          <h2 style={{margin:"0 0 2px",color:C.navy,fontSize:18,fontWeight:700}}>
          <p style={{margin:0,color:C.textLight,fontSize:13}}>{patients.filter(p=
        </div>
        <div style={{display:"flex",gap:8,flexWrap:"wrap"}}>
          {currentUser.role!==ROLES.EP&&<Btn onClick={onPrintAll} variant="print"
```

```
    <Btn onClick={()=>setAdding(true)} variant="primary">+ Add Patient</Btn
  </div>
</div>

<Card style={{padding:"12px 14px",marginBottom:14}}>
  <div style={{display:"flex",gap:8,flexWrap:"wrap",alignItems:"center"}}>
    <input value={q} onChange={e=>setQ(e.target.value)} placeholder="Search
      style={{...inputStyle,flex:1,minWidth:160,padding:"8px 11px"}}/>
    <div style={{display:"flex",gap:4}}>
      {["active","inactive","all"].map(f=>(
        <button key={f} onClick={()=>setFilter(f)}
          style={{padding:"7px 12px",borderRadius:5,border:`1px solid ${fil
          {f}
        </button>
      ))}
    </div>
    <div style={{display:"flex",gap:4}}>
      {["All","A","B","C","D"].map(l=>(
        <button key={l} onClick={()=>setLvl(l)}
          style={{padding:"7px 10px",borderRadius:5,border:`1px solid ${lvl
          {l==="All"?"All":l}
        </button>
      ))}
    </div>
    {currentUser.role===ROLES.ADMIN&&(
      <div style={{display:"flex",gap:4,flexWrap:"wrap"}}>
        <button onClick={()=>setSiteFilter("All")}
          style={{padding:"7px 10px",borderRadius:5,border:`1px solid ${sit
          All Sites
        </button>
        {sites.map(s=>(
          <button key={s.id} onClick={()=>setSiteFilter(s.id)}
            style={{padding:"7px 10px",borderRadius:5,border:`1px solid ${s
            {s.name}
          </button>
        ))}
      </div>
    )}
  </div>
</Card>

{vis.length===0
  ?<Card><p style={{margin:0,color:C.textLight,textAlign:"center"}}>No pati
  :vis.map(p=>{
    const lc=LEVEL_CONFIG[p.level];
    const ptS=sessions[p.id]||[];
    const last=ptS.length>0?[...ptS].sort((a,b)=>b.date.localeCompare(a.dat
```

```
          return (
            <div key={p.id} onClick={()=>onSelect(p.id)}
              style={{border:`1px solid ${C.border}`,borderLeft:`4px solid ${lc.c
              onMouseEnter={e=>e.currentTarget.style.boxShadow="0 2px 10px rgba(0
              onMouseLeave={e=>e.currentTarget.style.boxShadow="none"}>
              <div style={{display:"flex",justifyContent:"space-between",alignIte
                <div>
                  <div style={{display:"flex",gap:8,alignItems:"center",marginBot
                    <span style={{fontWeight:700,color:C.text,fontSize:15}}>{p.na
                    <LevelTag level={p.level}/>
                    {!p.active&&<span style={{fontSize:10,background:"#f0f0f0",co
                  </div>
                  <div style={{fontSize:12,color:C.textLight,display:"flex",gap:1
                    {currentUser.role===ROLES.ADMIN&&sites&&<span style={{fontWei
                    {p.room&&<span>Room {p.room}</span>}
                    {p.physician&&<span>Dr. {p.physician}</span>}
                    <span>{ptS.length} session{ptS.length!==1?"s":""}</span>
                    {last&&<span>Last: {last.date}</span>}
                  </div>
                </div>
                <span style={{color:C.textLight,fontSize:16}}>›</span>
              </div>
            </div>
          );
        })
      }
    </div>
  );
}

// ── DASHBOARD ──────────────────────────────────────────────────────
function Dashboard({ patients, sessions, currentUser, sites, onSessionClick }) {
  const active=patients.filter(p=>p.active);
  const lc={A:0,B:0,C:0,D:0};
  active.forEach(p=>lc[p.level]=(lc[p.level]||0)+1);
  const allS=Object.values(sessions).flat();
  const today=new Date().toISOString().split("T")[0];
  const site=sites.find(s=>s.id===currentUser.siteId);

  return (
    <div>
      <div style={{marginBottom:20}}>
        <h2 style={{margin:"0 0 2px",color:C.navy,fontSize:18,fontWeight:700}}>Da
        <p style={{margin:0,color:C.textLight,fontSize:13}}>{site?.name} — {new D
      </div>

      <div style={{display:"grid",gridTemplateColumns:"repeat(auto-fill,minmax(15
```

```
  <StatCard label="Active Patients" value={active.length}/>
  <StatCard label="Sessions Today" value={allS.filter(s=>s.date===today).le
  <StatCard label="This Week" value={allS.filter(s=>(new Date()-new Date(s.
  <StatCard label="Total Sessions" value={allS.length}/>
</div>

<div style={{display:"grid",gridTemplateColumns:"repeat(auto-fill,minmax(30
  <Card>
    <SectionTitle>Patients by Functional Level</SectionTitle>
    <DataTable
      headers={["Level","Description","Patients"]}
      rows={Object.entries(LEVEL_CONFIG).map(([l,c])=>[
        <LevelTag level={l}/>,
        <span style={{fontSize:12,color:C.textMid}}>{c.desc}</span>,
        <span style={{fontWeight:700,color:C.text}}>{lc[l]}</span>
      ])}
    />
  </Card>

  <Card>
    <SectionTitle>Recent Sessions</SectionTitle>
    {allS.length===0
      ?<p style={{margin:0,color:C.textLight,fontSize:13}}>No sessions logg
      :<div>
        {[...allS].sort((a,b)=>b.date.localeCompare(a.date)).slice(0,6).map
          const pt=patients.find(p=>(sessions[p.id]||[]).some(ss=>ss.id===s
          return (
            <div key={s.id} onClick={()=>onSessionClick&&onSessionClick(pt?
              style={{display:"flex",justifyContent:"space-between",alignIt
              onMouseEnter={e=>{if(onSessionClick)e.currentTarget.style.bac
              onMouseLeave={e=>e.currentTarget.style.background="transparen
              <div style={{display:"flex",gap:10,alignItems:"center",flex:1
                <LevelTag level={s.level}/>
                <div style={{minWidth:0}}>
                  <div style={{fontWeight:600,color:C.text,fontSize:13,whit
                  <div style={{fontSize:11,color:C.textLight}}>{s.clinician
                </div>
              </div>
              <div style={{textAlign:"right",flexShrink:0}}>
                <div style={{fontSize:12,color:C.textLight}}>{s.date}</div>
                {onSessionClick&&<div style={{fontSize:10,color:C.teal,font
              </div>
            </div>
          );
        })}
      </div>
    }
```

```
        </Card>
      </div>
    </div>
  );
}

// ── AUDIT LOG ──────────────────────────────────────────────────────────
function AuditLog({ currentUser, sites }) {
  const [log,setLog]=useState([]);
  useEffect(()=>{ load("kin_audit").then(d=>setLog(d||[])); },[]);
  const visible=log.filter(e=>currentUser.role===ROLES.ADMIN||e.siteId===currentU
  const roleColors={admin:C.danger,supervisor:C.warning,ep:C.teal};

  return (
    <div>
      <h2 style={{margin:"0 0 16px",color:C.navy,fontSize:18,fontWeight:700}}>Aud
      <Card style={{padding:0,overflow:"hidden"}}>
        <DataTable
          headers={["Timestamp","User","Role","Site","Action","Detail"]}
          rows={visible.slice(0,100).map(e=>[
            <span style={{fontSize:11,color:C.textLight,whiteSpace:"nowrap"}}>{ne
            <span style={{fontWeight:600,color:C.text}}>{e.username}</span>,
            <span style={{background:roleColors[e.role]||"#999",color:"#fff",bord
            <span style={{color:C.textMid,fontSize:12}}>{sites.find(s=>s.id===e.s
            <span style={{background:C.bg,border:`1px solid ${C.border}`,borderRa
            <span style={{color:C.textMid,fontSize:12}}>{e.detail}</span>
          ])}
          emptyMsg="No audit entries yet."
        />
      </Card>
    </div>
  );
}

// ── USER MANAGEMENT ────────────────────────────────────────────────────
function UserManagement({ users, sites, currentUser, onUpdateUsers, onAuditAction
  const [adding,setAdding]=useState(false);
  const [f,setF]=useState({username:"",password:"",name:"",role:ROLES.EP,siteId:c
  const s=k=>v=>setF(x=>({...x,[k]:v}));
  const roleColors={admin:C.danger,supervisor:C.warning,ep:C.teal};

  const save=()=>{
    if(!f.username||!f.password||!f.name)return alert("All fields required");
    if(users.find(u=>u.username===f.username))return alert("Username already exis
    onUpdateUsers([...users,{...f,id:Date.now().toString()}]);
    onAuditAction("USER_CREATED",`Created: ${f.username} (${f.role})`);
    setAdding(false);
```

```
    setF({username:"",password:"",name:"",role:ROLES.EP,siteId:currentUser.siteId
  };

  return (
    <div>
      <div style={{display:"flex",justifyContent:"space-between",alignItems:"cent
        <h2 style={{margin:0,color:C.navy,fontSize:18,fontWeight:700}}>User Manag
        <Btn onClick={()=>setAdding(a=>!a)} variant="primary">+ Add User</Btn>
      </div>
      {adding&&(
        <Card>
          <SectionTitle>New User</SectionTitle>
          <div style={{display:"grid",gridTemplateColumns:"repeat(auto-fill,minma
            <Input label="Full Name" value={f.name} onChange={s("name")} required
            <Input label="Username" value={f.username} onChange={s("username")} r
            <Input label="Password" value={f.password} onChange={s("password")} r
            <Select label="Role" value={f.role} onChange={s("role")} options={[{v
            <Select label="Site" value={f.siteId} onChange={s("siteId")} options=
          </div>
          <div style={{display:"flex",gap:8}}>
            <Btn onClick={save} variant="success">Save User</Btn>
            <Btn onClick={()=>setAdding(false)} variant="secondary">Cancel</Btn>
          </div>
        </Card>
      )}
      <Card style={{padding:0,overflow:"hidden"}}>
        <DataTable
          headers={["Name","Username","Role","Site","Status","Actions"]}
          rows={users.map(u=>[
            <span style={{fontWeight:600,color:C.text}}>{u.name}</span>,
            <span style={{color:C.textMid}}>{u.username}</span>,
            <span style={{background:roleColors[u.role]||"#999",color:"#fff",bord
            <span style={{color:C.textMid,fontSize:12}}>{sites.find(s=>s.id===u.s
            <span style={{background:u.active?C.success:C.textLight,color:"#fff",
            <div style={{display:"flex",gap:5}}>
              <Btn onClick={()=>{const pw=prompt("New password:");if(!pw)return;o
              {u.id!==currentUser.id&&<Btn onClick={()=>{onUpdateUsers(users.map(
            </div>
          ])}
        />
      </Card>
    </div>
  );
}

// ── SITE MANAGEMENT ──────────────────────────────────────────────
function SiteManagement({ sites, onUpdateSites, onAuditAction }) {
```

```
  const [adding,setAdding]=useState(false);
  const [f,setF]=useState({name:"",address:""});
  return (
    <div>
      <div style={{display:"flex",justifyContent:"space-between",alignItems:"cent
        <h2 style={{margin:0,color:C.navy,fontSize:18,fontWeight:700}}>Site Manag
        <Btn onClick={()=>setAdding(a=>!a)} variant="primary">+ Add Site</Btn>
      </div>
      {adding&&(
        <Card>
          <SectionTitle>New Site</SectionTitle>
          <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:"0 16px"}
            <Input label="Site Name" value={f.name} onChange={v=>setF(x=>({...x,n
            <Input label="Address" value={f.address} onChange={v=>setF(x=>({...x,
          </div>
          <div style={{display:"flex",gap:8}}>
            <Btn onClick={()=>{if(!f.name)return alert("Name required");onUpdateS
            <Btn onClick={()=>setAdding(false)} variant="secondary">Cancel</Btn>
          </div>
        </Card>
      )}
      <div style={{display:"grid",gridTemplateColumns:"repeat(auto-fill,minmax(24
        {sites.map(s=>(
          <Card key={s.id} style={{marginBottom:0}}>
            <div style={{fontWeight:700,color:C.navy,marginBottom:4,fontSize:14}}
            <div style={{fontSize:12,color:C.textLight}}>{s.address||"No address
          </Card>
        ))}
      </div>
    </div>
  );
}

// ── MAIN APP ──────────────────────────────────────────────────────────
export default function App() {
  const [patients,setPatients]=useState([]);
  const [sessions,setSessions]=useState({});
  const [users,setUsers]=useState(DEFAULT_USERS);
  const [sites,setSites]=useState(DEFAULT_SITES);
  const [currentUser,setCurrentUser]=useState(null);
  const [loginError,setLoginError]=useState("");
  const [view,setView]=useState("dashboard");
  const [selPt,setSelPt]=useState(null);
  const [focusSession,setFocusSession]=useState(null);
  const [loaded,setLoaded]=useState(false);

  useEffect(()=>{
```

```
  (async()=>{
    const p=await load("kin_patients"); const s=await load("kin_sessions");
    const u=await load("kin_users");    const si=await load("kin_sites");
    if(p)setPatients(p); if(s)setSessions(s);
    if(u)setUsers(u); else save("kin_users",DEFAULT_USERS);
    if(si)setSites(si); else save("kin_sites",DEFAULT_SITES);
    setLoaded(true);
  })();
},[]);

useEffect(()=>{if(loaded)save("kin_patients",patients);},[patients,loaded]);
useEffect(()=>{if(loaded)save("kin_sessions",sessions);},[sessions,loaded]);
useEffect(()=>{if(loaded)save("kin_users",users);},[users,loaded]);
useEffect(()=>{if(loaded)save("kin_sites",sites);},[sites,loaded]);

const doAudit=useCallback((action,detail="")=>{ if(currentUser) audit(currentUs

const handleLogin=(u,p)=>{
  const user=users.find(x=>x.username===u&&x.password===p&&x.active);
  if(!user){setLoginError("Invalid username or password.");return;}
  setCurrentUser(user); setLoginError("");
  audit(user,"LOGIN",`Signed in`);
};

const handleLogout=(reason="manual")=>{
  if(currentUser) audit(currentUser,reason==="timeout"?"TIMEOUT":"LOGOUT","Sess
  setCurrentUser(null); setView("dashboard"); setSelPt(null);
};

const sitePts=currentUser?.role===ROLES.ADMIN?patients:patients.filter(p=>p.sit
const selPatient=sitePts.find(p=>p.id===selPt);

const addPt=p=>{setPatients(ps=>[...ps,p]);doAudit("PATIENT_ADDED",`Added: ${p.
const editPt=p=>{setPatients(ps=>ps.map(x=>x.id===p.id?p:x));doAudit("PATIENT_E
const updLv=(id,lv)=>{const pt=patients.find(p=>p.id===id);setPatients(ps=>ps.m
const addS=useCallback((pid,s)=>{setSessions(ss=>({...ss,[pid]:[...(ss[pid]||[]
const delS=(pid,sid)=>{setSessions(ss=>({...ss,[pid]:(ss[pid]||[]).filter(s=>s.

const handlePrint=(pt,ptS)=>{printReport(pt,ptS,currentUser,sites);doAudit("REP
const printAll=()=>sitePts.filter(p=>p.active).forEach((p,i)=>setTimeout(()=>ha

const canAdmin=currentUser?.role===ROLES.ADMIN;
const canSuper=currentUser?.role!==ROLES.EP;

const NAV=[
  {id:"dashboard",label:"Dashboard",icon:"▣"},
  {id:"roster",label:"Patients",icon:"◉"},
```

```
  ...(canSuper?[{id:"audit",label:"Audit Log",icon:"≡"}]:[]),
  ...(canAdmin?[{id:"users",label:"Users",icon:"⊕"},{id:"sites",label:"Sites",i
];

if(!loaded) return <div style={{minHeight:"100vh",display:"flex",alignItems:"ce
if(!currentUser) return <Login onLogin={handleLogin} error={loginError}/>;

return (
  <InactivityGuard onLogout={handleLogout}>
    <div style={{display:"flex",flexDirection:"column",minHeight:"100vh",fontFa

      {/* TOP BAR */}
      <div style={{background:C.navy,padding:"0 16px",display:"flex",alignItems
        <div style={{display:"flex",alignItems:"center",gap:10}}>
          <div style={{width:3,height:24,background:C.teal,borderRadius:2}}/>
          <div>
            <div style={{fontSize:15,fontWeight:800,color:"#fff",letterSpacing:
            <div style={{fontSize:9,color:"rgba(255,255,255,0.35)",letterSpacin
          </div>
        </div>
        <nav style={{display:"flex",gap:2}}>
          {NAV.map(n=>(
            <button key={n.id} onClick={()=>{setView(n.id);setSelPt(null);}}
              style={{padding:"7px 12px",borderRadius:5,border:"none",cursor:"p
              {n.label}
            </button>
          ))}
        </nav>
        <div style={{display:"flex",alignItems:"center",gap:10}}>
          <div style={{textAlign:"right"}}>
            <div style={{fontSize:12,color:"#fff",fontWeight:600}}>{currentUser
            <div style={{fontSize:10,color:"rgba(255,255,255,0.4)",textTransfor
          </div>
          <button onClick={()=>handleLogout("manual")} style={{background:"rgba
        </div>
      </div>

      {/* CONTENT */}
      <div style={{flex:1,padding:"18px 16px",overflowY:"auto",paddingBottom:72
        {view==="dashboard"&&<Dashboard patients={sitePts} sessions={sessions}
        {view==="roster"&&(
          selPatient
            ?<PatientDetail patient={selPatient} sessions={sessions[selPt]||[]}
            :<Roster patients={sitePts} sessions={sessions} onSelect={id=>{setS
        )}
        {view==="audit"&&canSuper&&<AuditLog currentUser={currentUser} sites={s
        {view==="users"&&canAdmin&&<UserManagement users={users} sites={sites}
```

```
          {view==="sites"&&canAdmin&&<SiteManagement sites={sites} onUpdateSites=
        </div>

        {/* BOTTOM NAV */}
        <div style={{position:"fixed",bottom:0,left:0,right:0,background:C.navy,d
          {NAV.map(n=>(
            <button key={n.id} onClick={()=>{setView(n.id);setSelPt(null);}}
              style={{flex:1,padding:"10px 4px 8px",border:"none",cursor:"pointer
              <span style={{fontSize:14}}>{n.icon}</span>
              <span>{n.label}</span>
            </button>
          ))}
        </div>
      </div>
    </InactivityGuard>
  );
}
```